

3D Gaussian Splatting Scene Reconstruction

Nabeela Masroor
University of Genoa

Abstract

This project implements a complete 3D Gaussian Splatting (3DGS) pipeline using a custom dataset of 115 iPhone-captured images of a computer mouse. Using COLMAP for camera pose estimation and OpenSplat for training on Apple M1 hardware, we produce a 3D scene representation of 199,545 Gaussian primitives, visualised and post-processed in the SuperSplat editor.

1 Introduction

3D Gaussian Splatting [1] is a real-time scene reconstruction technique that represents a scene as a collection of explicit 3D Gaussian primitives, each defined by its position, shape, opacity, and view-dependent colour. Compared to NeRF-based methods [2], 3DGS offers significantly faster training and real-time rendering, making it well-suited for AR/VR applications.

In this project, we build a 3DGS reconstruction pipeline from scratch using only open-source tools on a consumer Apple M1 laptop. The dataset was self-captured using an iPhone 15, and every stage of the pipeline — from data capture and Structure-from-Motion to Gaussian optimisation and 3D visualisation — was implemented and run locally.

2 Related Work

NeRF [2] models a scene as a continuous implicit function, mapping spatial coordinates and viewing direction to colour and density through a neural network. While it produces high-quality novel views, it requires hours of training and is too slow for real-time rendering. 3DGS [1] addresses this by using explicit Gaussians that can be rendered via fast tile-based rasterisation, achieving over 100 FPS at 1080p on NVIDIA hardware.

For this project we used OpenSplat [4], a portable C++ implementation of 3DGS that supports not only NVIDIA CUDA but also Apple Metal (MPS) and CPU backends, mak-

ing it the only practical choice for Apple Silicon hardware without a dedicated NVIDIA GPU.

3 Methodology

The pipeline follows four sequential stages: dataset capture, camera pose estimation with COLMAP, Gaussian Splatting training with OpenSplat, and visualisation with SuperSplat.

3.1 Dataset Capture

The scene consists of a black wireless computer mouse placed on a colourful decorative carpet. The carpet was deliberately chosen as a background because its rich colour and texture provide abundant SIFT feature points for COLMAP. Images were captured with an iPhone 15 in standard JPEG mode (4284×5712 px). 115 photos were taken around the stationary object in three circular passes at low ($\sim 30^\circ$), mid ($\sim 90^\circ$), and high ($\sim 120^\circ$) elevation angles to ensure full coverage. Overlapping adjacent frames are essential for COLMAP: triangulation requires the same 3D point to appear in at least two images, and the more overlap there is, the more constrained and accurate the recovered geometry.

3.2 Structure-from-Motion with COLMAP

Before Gaussian Splatting can begin, we need to know where each camera was positioned when each photo was taken. COLMAP [3] recovers this through Structure-from-Motion (SfM) — a process of finding shared visual features across images and using their geometry to simultaneously estimate 3D point positions and camera poses.

Feature Extraction. SIFT (Scale-Invariant Feature Transform) keypoints are extracted from every image. SIFT was chosen because it is invariant to scale and rotation, making it robust to the varying camera angles used during capture. On average approximately 10,000 keypoints were detected per image. Each keypoint is described by a 128-dimensional descriptor vector that captures the local gradient structure around it.

Feature Matching. Exhaustive matching was used, meaning every possible image pair is compared. COLMAP finds descriptor matches between pairs and applies geometric verification to reject false matches — only correspondences that are geometrically consistent across views are kept. Exhaustive matching was chosen over faster alternatives such as vocabulary-tree matching because our dataset is small (115 images), and we wanted to maximise the number of valid correspondences.

Sparse Reconstruction. Using the verified matches, incremental SfM registers cam-

eras one by one into a growing 3D model. Each new camera’s position and orientation is estimated from its shared feature points with already-registered cameras. Bundle adjustment is run periodically to jointly refine all camera poses and 3D point positions, minimising the total reprojection error across the entire reconstruction. COLMAP successfully registered all 115 images, recovering **21,863 sparse 3D points** that serve as the initialisation for Gaussian Splatting.

```
colmap feature_extractor --database_path colmap/database.db
--image_path images --ImageReader.single_camera 1
colmap exhaustive_matcher --database_path colmap/database.db
colmap mapper --database_path colmap/database.db
--image_path images --output_path colmap/sparse
```

3.3 3D Gaussian Splatting with OpenSplat

3DGS represents the scene as a set of N 3D Gaussians. Each Gaussian is parameterised by a mean position $\boldsymbol{\mu} \in R^3$, a covariance matrix $\boldsymbol{\Sigma}$ encoding its shape and orientation, an opacity α , and spherical harmonic coefficients for view-dependent colour. The covariance is decomposed as:

$$\boldsymbol{\Sigma} = \mathbf{R}\mathbf{S}\mathbf{S}^\top\mathbf{R}^\top \quad (1)$$

where $\mathbf{R}$ is a rotation matrix and $\mathbf{S}$ is a diagonal scaling matrix. This decomposition keeps $\boldsymbol{\Sigma}$ positive semi-definite and allows the optimiser to freely adjust the shape and orientation of each Gaussian without producing invalid covariance matrices.

To render a novel view, the 3D Gaussians are projected onto the image plane and composited front-to-back. Each pixel colour C is computed as:

$$C = \sum_i c_i \alpha_i \prod_{j < i} (1 - \alpha_j) \quad (2)$$

where c_i is the colour of Gaussian i and the product term accumulates transmittance — Gaussians further from the camera are occluded by those in front. This is the same alpha compositing used in volume rendering, but applied to explicit Gaussian primitives instead of a sampled ray, which is what makes 3DGS fast compared to NeRF.

Training optimises all Gaussian parameters end-to-end using a photometric loss comparing rendered images to the input photos:

$$\mathcal{L} = (1 - \lambda)\mathcal{L}_1 + \lambda\mathcal{L}_{\text{D-SSIM}} \quad (3)$$

where $\mathcal{L}_1$ measures pixel-wise colour error and $\mathcal{L}_{\text{D-SSIM}}$ penalises structural dissimilarity. We use $\lambda = 0.2$, the default from the original paper, which balances per-pixel accuracy

with perceptual quality. Throughout training, Gaussians are also adaptively densified — split or cloned — in regions where the reconstruction is under-represented, and pruned where they become transparent or redundant. After 7,000 iterations the loss had converged to around 0.03, indicating a stable reconstruction.

OpenSplat was compiled from source on macOS with Apple Metal Toolchain support. Initially, training was attempted using the Metal (MPS) GPU backend; however, this caused memory errors when loading full-resolution iPhone 15 images (4284×5712 px) due to the 8 GB shared RAM of the M1. Training was therefore run in CPU mode with a downscale factor of 2, reducing the effective resolution by half. We ran 7,000 iterations, the standard fast-training setting from the original 3DGS paper, balancing reconstruction quality and compute time.

```
./opensplat colmap/ -n 7000 --cpu -d 2
```

Training took approximately 33 hours, producing a `splat.ply` file with **199,545 Gaussian primitives**.

3.4 Visualisation and Post-processing

The resulting `splat.ply` was loaded into the SuperSplat web editor. The reconstruction contained floating Gaussian artefacts at the scene boundary — a known issue in unbounded 3DGS scenes where Gaussians are placed to explain background pixels that were not consistently observed across views. These were removed using lasso selection in SuperSplat.

4 Hardware and Software

Component	Details
Hardware	Apple MacBook Air (M1, 8 GB RAM)
OS	macOS Tahoe 26.3
Camera	iPhone 15
COLMAP	v4.0.4 (Homebrew)
OpenSplat	v1.1.5 (built from source)
Viewer	SuperSplat Editor v2.27.3

Table 1: Hardware and software used in this project.

5 Results

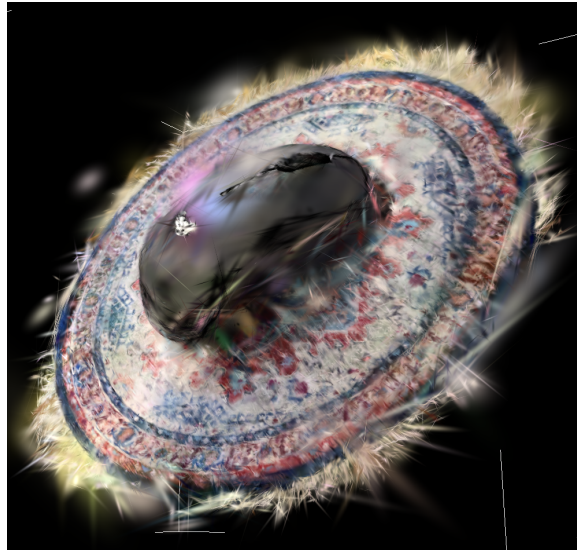
The pipeline successfully reconstructed the scene, producing a 3D Gaussian model with 199,545 primitives.



(a) Top-down view



(b) Close-up diagonal view



(c) Side/tilted view

Figure 1: Reconstruction rendered from three viewpoints in SuperSplat. The overall shape of the mouse and the carpet texture are consistently visible across views.



(a) Before post-processing



(b) After post-processing

Figure 2: Before and after post-processing. Floating Gaussian artefacts around the scene boundary were removed using lasso selection in SuperSplat.

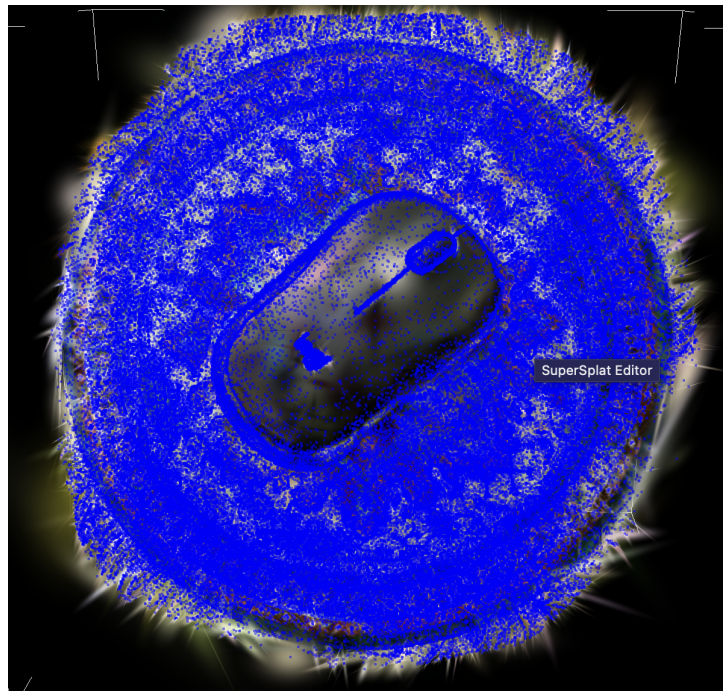


Figure 3: Spatial distribution of 199,545 Gaussian primitives. Each blue dot represents one Gaussian centre, illustrating how densely the scene is covered.

The overall geometry of the mouse is well recovered and recognisable from all viewpoints. The carpet background, with its dense colour and texture, reconstructs with particularly good fidelity.

6 Discussion and Limitations

The most significant constraint of was hardware. The Apple M1’s 8 GB of shared memory was insufficient for OpenSplat’s Metal GPU backend at full image resolution, forcing a fallback to CPU execution with a downscale factor of 2. This increased training time from what would typically be 15–30 minutes on an NVIDIA GPU to approximately 33 hours. The downscaling also likely reduced fine detail in the reconstruction. Running more iterations would further improve Gaussian placement but was not feasible within the available compute time.

7 Conclusion

This project demonstrates a complete 3DGS reconstruction pipeline running entirely on consumer Apple Silicon hardware without any NVIDIA GPU. Starting from 115 self-captured iPhone images, the pipeline produces a 199,545-Gaussian model of a real-world scene that is visually coherent and freely explorable in 3D.

References

- [1] Kerbl, B., Kopanas, G., Leimkühler, T., & Drettakis, G. (2023). *3D Gaussian Splatting for Real-Time Radiance Field Rendering*. ACM SIGGRAPH 2023. <https://github.com/graphdeco-inria/gaussian-splatting>
- [2] Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., & Ng, R. (2020). *NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis*. ECCV 2020.
- [3] Schönberger, J. L., & Frahm, J. M. (2016). *Structure-from-Motion Revisited*. CVPR 2016. <https://colmap.github.io>
- [4] Pierotofy. (2024). *OpenSplat: Production-grade 3D Gaussian Splatting*. <https://github.com/pierotofy/OpenSplat>
- [5] PlayCanvas. (2024). *SuperSplat: 3D Gaussian Splat Editor*. <https://playcanvas.com/supersplat/editor>